

StudySync

PWA de Gestion Academica

Guia de desarrollo por modulos

Electiva III — 2026

1. Estado actual del proyecto

La Persona 1 ya entrego la base completa. Todo lo que sigue construye sobre esto.

Lo que ya esta listo

Persona 1 — Base completada
• Servidor Express en puerto 3000 con CORS y soporte base64 (50mb)
• API REST: GET /api, POST /api, DELETE /api/:id, GET /api/materias
• Filtros por materia, tipo y busqueda por texto en el servidor
• UI completa: pantalla de perfil, navbar, timeline, modal, filtros y busqueda en tiempo real
• Modo oscuro con persistencia en localStorage
• Exportar todos los apuntes a PDF con portada y formato profesional
• Service Worker base y manifest.json para instalacion como PWA
• Modelo de datos definido y documentado en MODELO_DATOS.md

Modelo de datos — leer antes de escribir codigo

Definido en MODELO_DATOS.md en la raiz del repo. Todos deben respetar estos campos:

- _id: string ISO date (generado automatico con new Date().toISOString())
- tipo: "apunte" | "evento" — obligatorio
- titulo: string — obligatorio
- contenido: string
- materia: string — obligatorio
- tags: string[] — ej: ["#parcial", "#quiz"]
- fechaCreacion: string ISO date (generado automatico)
- fechaEntrega: string ISO date | null — solo para eventos
- foto / audio / video: string base64 | null — Persona 2 los llena
- lat / lng: number | null — Persona 2 los llena
- user: string — obligatorio

2. Persona 2 — Recursos Nativos

INDEPENDIENTE — no depende de Persona 3 ni 4

Conecta camara, microfono, video y geolocalizacion al modal. Los botones ya existen en index.html.

Archivos que debes tocar

- public/js/camara-class.js — ampliar con metodos de grabacion de audio
- public/js/app.js — conectar los botones (buscar el comentario "hooks para Persona 2")

Archivos que NO debes tocar

- public/index.html
- public/css/style.css
- server/server.js
- server/routes.js

Paso a paso

#	Paso	Detalle
1	Clonar y levantar	git clone, npm install, npm run dev — verificar http://localhost:3000
2	Leer MODELO_DATOS.md	Entender los campos: foto, audio, video, lat, lng
3	Verificar foto	btnFoto ya abre camara. tomarFoto() guarda en adjuntos.foto en base64
4	Verificar video	btnVideo ya graba. detenerGrabacion() guarda en adjuntos.video
5	Implementar audio	Agregar iniciarGrabacionAudio() y detenerGrabacionAudio() en camara-class.js usando MediaRecorder con audio:true
6	Conectar audio	En app.js buscar btnAudio.on("click") y reemplazar el toast por la logica real
7	Verificar ubicacion	btnUbicacion ya funciona. Verificar que guarda lat/lng y el mapa aparece en la tarjeta
8	Probar flujo	Crear apuntes con foto, con ubicacion y con audio. Ver que se muestran en el timeline

Implementacion sugerida para audio en camara-class.js

```
iniciarGrabacionAudio() {
  navigator.mediaDevices.getUserMedia({ audio: true, video: false })
    .then(stream => {
      this.audioStream = stream;
      this.audioChunks = [];
      this.audioRecorder = new MediaRecorder(stream);
      this.audioRecorder.ondataavailable = e => this.audioChunks.push(e.data);
      this.audioRecorder.start();
    });
}

detenerGrabacionAudio() {
```

```
return new Promise((resolve, reject) => {
  this.audioRecorder.onstop = () => {
    const blob = new Blob(this.audioChunks, { type: "audio/webm" });
    const reader = new FileReader();
    reader.readAsDataURL(blob);
    reader.onloadend = () => resolve(reader.result);
  };
  this.audioRecorder.stop();
  this.audioStream.getTracks().forEach(t => t.stop());
});
}
```

3. Persona 3 — Offline y Service Worker

INDEPENDIENTE — no depende de Persona 2 ni 4

PouchDB para persistencia offline, Background Sync para sincronizar al volver la conexion e indicador visual online/offline.

Archivos que debes tocar

- public/sw.js — ampliar logica de sync
- public/js/sw-db.js — adaptar guardarMensaje() al modelo de apuntes
- public/js/sw-utils.js — ya funciona, revisar manejoApiMensajes()
- server/routes.js — agregar persistencia en archivo JSON

Archivos que NO debes tocar

- public/index.html
- public/css/style.css
- public/js/app.js
- server/server.js

Paso a paso

#	Paso	Detalle
1	Clonar y levantar	git clone, npm install, npm run dev
2	Leer MODELO_DATOS.md	El _id debe generarse con new Date().toISOString() en PouchDB
3	Adaptar sw-db.js	guardarMensaje(apunte) recibe el objeto completo. Solo asegurar que _id sea ISO date
4	Verificar Background Sync	sw.js tiene el listener "nuevo-post". Verificar que postearMensajes() hace POST a /api correctamente
5	Persistencia servidor	En routes.js leer/escribir apuntes-db.json con fs igual que subs-db.json
6	Banner offline	Agregar un div fijo visible cuando navigator.onLine es false. app.js ya tiene estadoConexion()
7	Probar offline	DevTools -> Network -> Offline. Crear apunte. Volver Online. Verificar que aparece en servidor

Persistencia en routes.js — implementacion sugerida

```
const DB_PATH = __dirname + "/apuntes-db.json";
function leerApuntes() {
  if (!fs.existsSync(DB_PATH)) return [];
  return JSON.parse(fs.readFileSync(DB_PATH, "utf8"));
}
function guardarApuntes(apuntes) {
  fs.writeFileSync(DB_PATH, JSON.stringify(apuntes, null, 2));
}
```

4. Persona 4 — Notificaciones y Push

DEPENDEN de Persona 1 (ya lista)

Notificaciones push con VAPID para recordatorios de eventos, notificaciones locales y manejo de clicks.

Archivos que debes tocar

- `server/push.js` — copiar del repo base ChatRecursosMoviles, ya es compatible
- `server/routes.js` — descomentar rutas `/subscribe`, `/key` y `/push`
- `public/sw.js` — ampliar el listener de push con icono del usuario
- `public/js/app.js` — activar botones de notificaciones

Archivos que NO debes tocar

- `public/index.html`
- `public/css/style.css`
- `server/server.js`

Paso a paso

#	Paso	Detalle
1	Clonar y levantar	git clone, npm install, npm run dev
2	Generar claves VAPID	npm run generate-vapid — genera <code>server/vapid.json</code>
3	Copiar <code>push.js</code>	Tomar <code>push.js</code> del repo base y pegarlo en <code>server/push.js</code> . Ya es compatible
4	Activar rutas	Las rutas <code>/subscribe</code> , <code>/key</code> y <code>/push</code> estan reservadas en <code>routes.js</code> con comentarios
5	Push al crear evento	En POST <code>/api</code> , si <code>tipo === "evento"</code> y tiene <code>fechaEntrega</code> llamar <code>push.sendPush()</code>
6	Ampliar <code>sw.js</code>	Agregar icono del avatar: <code>icon: "img/avatars/" + data.usuario + ".png"</code>
7	Notificaciones locales	Al cargar apuntes, revisar eventos con <code>fechaEntrega</code> menor a 3 dias y mostrar Notification local
8	Probar	Crear evento con fecha proxima. Verificar notificacion push. Probar click que abre la app

Uso del campo `fechaEntrega` en `routes.js`

```
if (apunte.tipo === "evento" && apunte.fechaEntrega) {
  const dias = Math.round(
    (new Date(apunte.fechaEntrega) - new Date()) / 86400000
  );
  push.sendPush({
    titulo: "Recordatorio - " + apunte.titulo,
    cuerpo: "Faltan " + dias + " dias para la entrega",
    usuario: apunte.user
  });
}
```

5. Git — Flujo de trabajo

Cada persona trabaja en su propia rama. Nadie sube directo a main.

Estructura de ramas

- main — version estable que siempre corre
- desarrollo — integracion antes de subir a main
- persona/persona1 — Persona 1 (ya lista para mergear)
- persona/persona2 — Persona 2
- persona/persona3 — Persona 3
- persona/persona4 — Persona 4

Comandos para el primer dia

#	Paso	Detalle
1	Clonar el repo	git clone
2	Crear tu rama	git checkout -b persona/persona2 (cambiar el numero)
3	Instalar y correr	npm install && npm run dev
4	Verificar	Abrir http://localhost:3000 — ver pantalla de perfil
5	Empezar a codear	Solo tocar los archivos de tu lista

Guardar y subir cambios

#	Paso	Detalle
1	Ver cambios	git status
2	Agregar	git add nombre-del-archivo.js (NO usar git add . a ciegas)
3	Commit	git commit -m "feat: descripcion corta"
4	Subir rama	git push origin persona/persona2
5	Pedir merge	Avisar al grupo para hacer PR a desarrollo

Reglas que todos deben respetar
• NUNCA hacer git push origin main directamente
• NUNCA modificar index.html, style.css ni app.js — si necesitas cambiar algo en esos archivos, crea un issue en el repo o avisa al grupo. Persona 1 decide si el cambio va o no
• SIEMPRE hacer git pull origin desarrollo antes de empezar cada dia
• Si hay conflicto al mergear, la persona duena del archivo lo resuelve
• Un commit = una cosa concreta. No mezclar funcionalidades distintas
• Mensajes de commit en minuscula: feat: / fix: / style: / docs:

Cuando todos terminen — integracion final

#	Paso	Detalle
1	Merge a desarrollo	Cada persona hace PR de su rama a desarrollo
2	Prueba integral	Con desarrollo corriendo, probar todas las funcionalidades juntas
3	Fix de conflictos	La persona duena del archivo que falla lo arregla
4	Merge a main	Solo cuando todo corre limpio se mergea a main
5	Tag de version	git tag v1.0.0 && git push origin v1.0.0

6. Resumen de responsabilidades

Persona	Modulo	Archivos principales	Depende de
Persona 1 (tu)	Base + UI + PDF	index.html, app.js, routes.js, style.css	Nadie
Persona 2	Recursos nativos	camara-class.js, app.js (hooks)	Persona 1
Persona 3	Offline + SW	sw.js, sw-db.js, routes.js	Persona 1
Persona 4	Notificaciones	push.js, routes.js, sw.js	Persona 1

Cualquier duda coordinan en el grupo antes de tocar archivos compartidos.